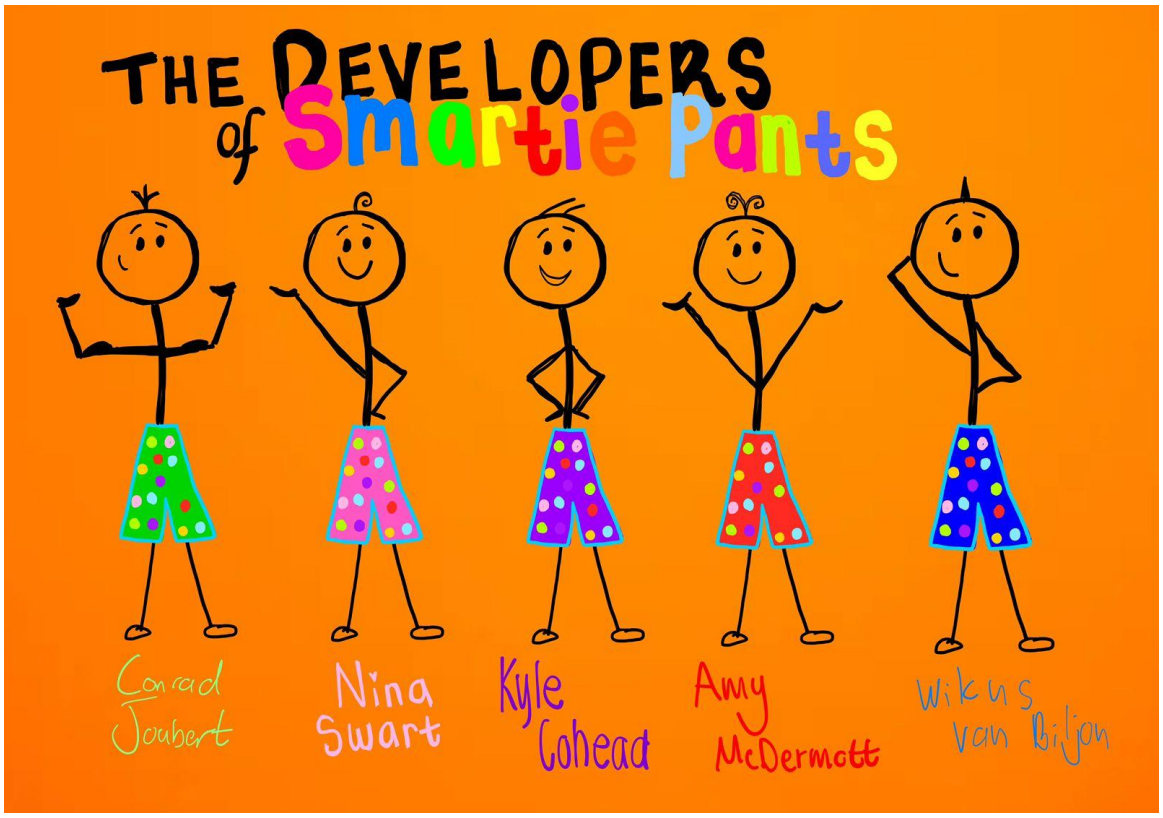
Group 7: SMARTIE PANTS



Multi-Player Trivia Tournament

Date of Submission: 19 October 2025

Computer Science 343 – Project 2
Stellenbosch University

Group Members:

Nina Swart	Student No. 26970341
Wikus van Biljon	Student No. 26927543
Amy Mc Dermott	Student No. 26911264
Conrad Joubert	Student No. 26989581
Kyle Cohead	Student No. 25964917

Table of Contents

1. Introduction ***1***

1.1 Project Overview **1**

2. Use Case Diagram & User Stories ***1***

2.1 Use Case Diagram 1

2.2 User Stories (Player) 1

2.3 User Stories (Admin) 2

3. Data Modelling ***2***

3.1 Entity-Relationship (ER) Diagram 2

3.2 Schema 2

3.3 Normalization to 3NF 3

4. Authentication ***4***

4.1 JWT Authentication 4

4.2 Password Hashing 4

5. API Design & WebSocket / Synchronization Events ***4***

5.1 REST API Endpoints 4

5.2 WebSocket Events 5

6. Design Patterns, Client and Server Architecture ***5***

6.1 Server Architecture (Backend) 5

6.2 Client Architecture (Frontend) 6

6.3 Design Patterns 6

7. Deployment & Environment ***6***

7.1 Deployment and Hosting 6

7.2 Major Dependencies 6

8. Extra Features, Known Issues, Contribution Log ***7***

8.1 Extra Features Implemented 7

8.2 Known Issues 8

8.3 Contribution Log 8

9. Contributions Detailing AI Usage, External Code, and Sources ***9***

9.1 AI/LLM Usage 9

9.2 External Code and Sources 9

References ***9***

Appendices ***10***

Appendix A: Use Case Diagram **10**

Appendix B: Entity Relationship Diagram **11**

Appendix C: Authentication Implementation **12**

Appendix D: API Endpoints **13**

Appendix E: WebSocket Event Reference **15**

Appendix F: Detailed Contribution Log ..... Error! Bookmark not defined.

1. Introduction

1.1 Project Overview

Smartie Pants is a multiplayer trivia web application developed as part of our Computer Science 343 coursework. The game draws inspiration from classic quiz tournaments, turning general knowledge into a fast-paced, social competition. Players can log in, join or host matches, and compete across timed rounds featuring categories such as Science, Entertainment, Sports, History, and General Knowledge.

Trivia questions are automatically collected through a custom web scraper, ensuring the question bank remains dynamic without any manual input. The platform includes both player and admin interfaces, allowing users to enjoy matches while admins manage questions, categories, and users.
The main goal of Smartie Pants is to create an engaging, real-time trivia experience that is both fun and technically robust. The project focuses on clean architecture, responsive design, and a smooth multiplayer system built using React (Vite) for the frontend, Node.js and Express for the backend, PostgreSQL for data storage, and Socket.IO for real-time communication. Ultimately, Smartie Pants combines knowledge, speed, and strategy to crown one player as "The Ultimate Smartie Pants."

2. Use Case Diagram & User Stories

2.1 Use Case Diagram

The system's functionality is summarized in the Use Case Diagram (see **Appendix A. Figure 1**). It outlines the interactions between two main actors — *Player* and *Admin* — and the core system components such as authentication, match management, question handling, and leaderboard viewing.

2.2 User Stories (Player)

As a Player, I want to create a profile (register), so that I can log in and participate in trivia games.
As a Player, I want to log in securely, so that I can access my account and personalized features.
As a Player, I want to edit my profile details (username, password, avatar), so that I can keep my information updated and personalized.
As a Player, I want to view my match history and stats (high score, games played, member since date), so that I can track my progress and performance.
As a Player, I want to delete my account, so that I can remove all my personal data and stop using the application.
As a Player (Host), I want to create a new match, so that I can set up a game for myself and my friends.
As a Player (Host), I want to choose categories and difficulty for the match rounds, so that I can customize the game content.
As a Player (Host), I want to invite other users to my match, so that they can join the game.
As a Player (Guest), I want to accept or reject a game invitation, so that I can decide whether to join an upcoming match.

As a Player, I want to view a list of all matches I've created or joined in "My Matches", so that I can easily track my upcoming and past games.
As a Player, I want to search and filter the list of upcoming/current matches by category, so that I can find games I'm interested in quickly.
As a Player, I want to see a synchronized countdown timer for each question, so that I know how much time I have left to answer.
As a Player, I want live score updates during the match, so that I can see my standing live standing.
As a Player, I want to see the correct answer after the time runs out, so that I can learn the answer.
As a Player, I want to see a post-match summary (ranking, points, average time), so that I can review my performance immediately after the game.

2.3 User Stories (Admin)

As an Admin, I want to view, search, and filter all stored trivia questions in the question bank, so that I can manage the content.
As an Admin, I want to be able to manually edit or delete existing questions, so that I can correct errors or remove inappropriate content.
As an Admin, I want to view, search, and manage categories, so that I can maintain the list of available categories.
As an Admin, I want to be able to delete users from the trivia application, so that I can manage user accounts.
As an Admin, I want to be able to delete users from trivia games, so that I can manage active matches.
As an Admin, I want to see a list of users currently in a match (real-time presence), so that I can monitor active games.

3. Data Modelling

3.1 Entity-Relationship (ER) Diagram

The database design for *Smartie Pants* is illustrated in the **Entity Relationship (ER) Diagram** (see **Appendix B**). It outlines the relationships between key entities such as *User*, *Match*, *Question*, *Category*, and *MatchPlayer*. The schema was normalized to **Third Normal Form (3NF)** to eliminate redundancy and ensure efficient data management. This structure supports core functionality including player authentication, match creation, question retrieval, and score tracking.

3.2 Schema

The logical schema below shows how the database tables are structured and linked to support the system's core features.

1. User

- Purpose: User specific data and gameplay aggregates.
- Key fields: id (PK), username, email, password, gamesPlayed, highScores, wins.
- Relationships: The user can host matches via Match.hostId; participates via MatchPlater.userId; submits Answer.userId; sends/receives notifications via Notification.senderId/receiverId.
- Normalisation: All non-key attributes depend on id. The metrics are kept consistent in application logic.

2. Match
    - Purpose: Game session metadata and lifecycle.
    - Key Fields: id (PK), title, category, optional difficulty, visibility/rules (isPublic=true, maxPlayers=5, timeLimit=10), status flags (status=LOBBY, completed=FALSE), timestamps (createdAt=now(), startedAt, finishedAt).
    - Relationships: A user can host many matches (1-to-many); each match has exactly one host. Each match has many match players, match questions and answers.
    - Normalisation: Attributes describe the match row only; no transitive dependencies.
3. Question
    - Purpose: Question bank entry.
    - Key Fields: id (pk), category, optional difficulty, question, correct.
    - Relationships: Each question appears in matches via MatchQuestion.questionId; every instance of Question has many instances of Answer.
    - Normalisation: All attributes depend on the question primary key with no partial or transitive dependencies.
4. MatchQuestion
    - Purpose: Join table binding questions to a match with ordering.
    - Key Fields: id (PK), matchId -> Match.id, questionId -> Question.id, order.
    - Constraints: A unique combination of matchId and questionId is enforced to prevent duplicate questions in a round.
    - Normalisation: Attributes depend on the composite key (matchId, questionId) with no partial or transitive dependencies.
5. matchPlayer
    - Purpose: Player membership and per-match state.
    - Key Fields: id (PK), matchId -> Match.id, userId -> User.id, joinedAt = now(), connected = true, score = 0.
    - Constraints: A unique combination of matchId and userId to enforce one membership row per user per match.
    - Normalisation: Non-key attributes depend on the membership row. No partial or transitive dependencies.
6. Notification
    - Purpose: In-app messaging for invites.
    - Key Fields: id (PK), type, status=PENDING, message, data.
    - Relationships: senderId -> User.id, receiverId -> User.id
    - Normalisation: All attributes describe the notification row with no transitive dependencies.
7. Answer
    - Purpose: A user's submitted answer to a question within a match.
    - Key Fields: id (PK), selected, isCorrect, createdAt=now().
    - Relationships: userId -> User.id, questionId -> Question.id, matchId -> Match.id.
    - Normalisation: All attributes depend on the answer row with no partial and transitive dependencies.

3.3 Normalization to 3NF

The database is designed according to Third Normal Form (3NF) principles to ensure consistency, integrity, and efficiency. In this structure, all attributes are atomic (1NF), every

non-key attribute depends on the whole primary key (2NF), and no non-key attribute depends on another non-key attribute (no transitive dependencies, 3NF). Each table — *User*, *Match*, *MatchPlayer*, *Question*, *MatchQuestion*, *Answer*, and *Notification* — contains only data relevant to its purpose, with primary and foreign keys defining clear relationships between entities. This separation removes redundancy, prevents data anomalies, and ensures that each fact is stored once within the database.

4. Authentication

The *Smartie Pants* application uses **JWT-based authentication** and **bcrypt password hashing** to ensure secure user access and data protection. Authentication and authorization are handled on the backend, while tokens maintain session state for logged-in users.

4.1 JWT Authentication

The system implements **access** and **refresh tokens** to manage sessions securely.

- **Access Token:** 30-minute lifespan for API requests.
- **Refresh Token:** 7-day lifespan for obtaining new access tokens.

Separate secrets are used for each token type, both stored in environment variables (ACCESS_TOKEN_SECRET, REFRESH_TOKEN_SECRET). Middleware verifies the JWT on protected routes to authorize user actions based on their ID and role. This setup ensures short-lived, renewable sessions that minimize the risk of token compromise.
(Implementation details in Appendix C.)

4.2 Password Hashing

All passwords are hashed using **bcrypt** before storage, with a **unique salt** and **10 salt rounds** per hash to resist brute-force attacks. Passwords are compared securely during login, and only the hashed values are stored in the database. This one-way encryption ensures that user credentials remain confidential even if the database is compromised.
The combination of JWT for stateless session management and bcrypt for secure password storage provides a robust, standards-based authentication system.

5. API Design & WebSocket / Synchronization Events

5.1 REST API Endpoints

The backend exposes REST API Endpoints build with Express.js. All routes follow a consistent /api/… structure and return a JSON response. Some routes require Authentication and are secured with JWT middleware. There are 6 main route directories that contain all the endpoints used:

1. **auth.js**: Signup, Login and refresh tokens
2. **users.js**: Get/update user profile, search users
3. **upload.js**: Avatar / image upload
4. **matches.js**: Create, list, join, get match details
5. **notifications.js**: Invites, accept/decline, mark read/delete
6. **questions.js**: Manage question bank, import/reset, random fetch

Only the authentication and upload routes are public; all others require a valid JWT access token. The full list of endpoints, grouped by module, is included in **Appendix D**.

5.2 WebSocket Events

Real-time gameplay is implemented using **Socket.IO**, enabling synchronized question delivery, live score updates, and instant feedback across all connected players. Each socket connection is authenticated using a **JWT token**, ensuring only valid users can join a match. If authentication fails, the connection request is rejected.

Two key authenticated events — **joinMatch** and **startMatch** — verify a user's identity and match access, while other real-time interactions are handled per active socket.

The system maintains an **in-memory match store** to track active matches, player scores, timers, and question state. Timers are synchronized server-side to ensure fairness, and scoring is based on response speed and accuracy.

A full list of **Socket.IO events**, their directions, and payload structures is provided in **Appendix E**.

6. Design Patterns, Client and Server Architecture

6.1 Server Architecture (Backend)

The backend is implemented in **Node.js** using **Express** for HTTP endpoints and **Socket.IO** for real-time match coordination. The codebase follows a clear separation of concerns: lightweight Express route handlers expose REST endpoints, **services** handle application logic, and **Prisma ORM** manages database access. Real-time gameplay (questions, scoring, leaderboards, player presence) is coordinated by a dedicated Socket.IO module that integrates with the same service and data layers to maintain consistent state across HTTP and WebSocket flows.

Key Components:

- **Routes (backend/src/routes/*.js):** Define API endpoints for sessions, matches, and admin actions. Route handlers validate input and delegate to service functions (e.g., matches.js, auth.js).
- **Services / Controllers:** Contain reusable domain logic for match creation, question selection, and scoring. Used by both REST and WebSocket layers to prevent duplication.
- **Database Layer (Prisma):** Interacts with **PostgreSQL** via the Prisma client (prisma/schema.prisma). Uses transactions for atomic updates such as awarding points or updating match state.
- **Socket Layer (backend/src/socket.js):** Handles real-time events like joinMatch, submitAnswer, and questionResults. Broadcasts state changes to clients while relying on the service layer for consistency.
- **Authentication & Security:** All requests are protected using **JWT-based middleware**. Socket connections must present a valid token before joining matches.

6.2 Client Architecture (Frontend)

The frontend is built with **React (Vite)** and styled using **Tailwind CSS**. The component structure follows a modular design, with dedicated components for login, match lobby, gameplay, leaderboard, and profile management.
Application state is managed using **React hooks** and **Context API** to share authentication and match data across components.
Real-time updates from Socket.IO are handled through event listeners that trigger re-renders without manual refreshes, ensuring a smooth and responsive user experience.

6.3 Design Patterns

Both the client and server follow a **modular, layered architecture**.

- **Backend:** follows a *service–controller–route* pattern, promoting separation of concerns and reusability.
- **Frontend:** implements the *container–presentational* pattern, where logic-heavy components manage data and pass it to lightweight UI components for rendering. State and data flows are unidirectional, simplifying debugging and maintaining predictable behavior across user interactions.

7. Deployment & Environment

7.1 Deployment and Hosting

The application is hosted on a **remote Ubuntu 22.04 LTS server** provided by **Hetzner Cloud**, which serves as the production environment for all services.
External access is secured and routed through a **Cloudflare Tunnel**, providing HTTPS connectivity without directly exposing the server's ports to the public internet.

The system uses **Docker Compose** for containerized deployment and development. This setup ensures environment consistency and easy redeployment by managing multiple containers — including the **backend, frontend,** and **PostgreSQL database** — within a single configuration file.

Both the **frontend and backend** are deployed on the **same server**, sharing the same Docker network. The backend communicates directly with the PostgreSQL service internally, while the frontend accesses the backend API through the Cloudflare-secured domain.

The deployed system is accessible via:
https://play.smartiepants.art

7.2 Major Dependencies

Technologies: React, Tailwind CSS, Node.js/Express, Socket.IO, PostgreSQL, Prisma, JWT/Bcrypt.

Category	Technology	Purpose
Frontend	React (Vite), Tailwind CSS	UI components and styling
Backend	Node.js, Express	REST API and WebSocket services
Database	PostgreSQL, Prisma ORM	Persistent data storage and schema management
Real-time	Socket.IO	Live gameplay and synchronization
Auth & Security	JWT, bcrypt	Authentication and password hashing
Hosting / Infrastructure	Hetzner, Cloudflare Tunnel	Remote hosting and secure public access
Containerized development	Docker / compose	Simplifies deployment and ensures consistent environments

8. Extra Features, Known Issues, Contribution Log

8.1 Extra Features Implemented

Feature	Description
Custom Stick-Man Character	Each player is represented by a stick-man avatar that can be customized in **height, width, thickness, and colour**, adding a playful and personal visual element to the game.
Hand-Drawn Styling	All visual elements were created in a **hand-drawn style** by group member **Nina Swart**, giving the game a unique and artistic appearance.
Email Invitations	Players can invite others via email to join matches. Each invitation includes a unique match link that opens directly in the browser. Implemented using the SendGrid API.
Browser Notifications	Players receive notifications for game invitations, match start alerts, and player activity. Implemented using the Web Notifications API.
Public Hosting	The full application is deployed on a **remote Ubuntu server** hosted by **Hetzner**, accessible externally via a **Cloudflare tunnel**.

Responsive UI Design	The interface adapts seamlessly to different screen sizes (desktop, tablet, mobile) using Tailwind CSS.
Live Leaderboard	Scores and player rankings update in real time during matches using Socket.IO events.
Dockerized Environment	The entire system is containerized using **Docker Compose**, allowing all components (frontend, backend, and database) to run consistently across development and production. This simplifies setup, deployment, and testing by eliminating environment-specific issues and ensuring reproducibility.

8.2 Known Issues

The Smartie Pants Trivia Game runs smoothly with no known bugs. Minor delays may occur during image loading or trivia data scraping, which are expected network-related behaviors.

8.3 Contribution Log

The project was developed collaboratively, with each member contributing to specific components according to their skills and interests. Responsibilities were divided across the frontend, backend, database, and design to maintain balanced workloads and steady progress. Regular GitLab commits, issue tracking, and team meetings supported coordination and followed agile development practices.

A summary of each member's primary role and key contributions is provided in this section. A complete record of daily commits, issue resolutions, and development activities is available in the **PROJECT_LOG.md** file within the main GitLab repository.

Member	**Student No.**	**Primary Role**	**Key Contributions**
Conrad Joubert	26989581	Game invites and authentication:	Send, receive, accept and decline invitations to games to and from players, Refresh token control
Nina Swart	26970341	UI/UX Designer & Frontend Stylist	Visual designs & backgrounds, initial setup, UI pages & profiling
Amy McDermott	26911264	Multifunctional & database	Leaderboard, game scheduling, match history
Kyle Cohead	25964917	Admin functionality and scraping	Socket.IO setup, live gameplay logic
Wikus van Biljon	26927543	Deployment, Infrastructure & Live gameplay	Login and signup, Docker, Socket.IO,

9. Contributions Detailing AI Usage, External Code, and Sources

9.1 AI/LLM Usage

In accordance with the *Computer Science 343 – Project 2 Specification*, **no AI-generated code** was used or included in the *Smartie Pants* project. Large Language Models (LLMs) such as ChatGPT were used **only for concept clarification and learning support**, not for direct code generation or design implementation.

AI assistance was limited to:

- Explaining programming concepts (e.g., Prisma schema syntax, Socket.IO event flow).
- Clarifying debugging approaches for minor runtime or logic issues.
- Reviewing documentation for grammar and clarity.

All frontend, backend, and database logic was written independently by the development team.

Artwork and Design:
All visual elements, including the **stick-man character**, **UI icons**, and **background artwork**, were **originally hand-drawn by Nina Swart**, a member of the group. No AI tools were used to generate any images or design assets.

9.2 External Code and Sources

The team adhered strictly to academic integrity requirements and used only **official documentation, open-source libraries, and public learning materials**. No third-party repositories, example projects, or copied code were used.

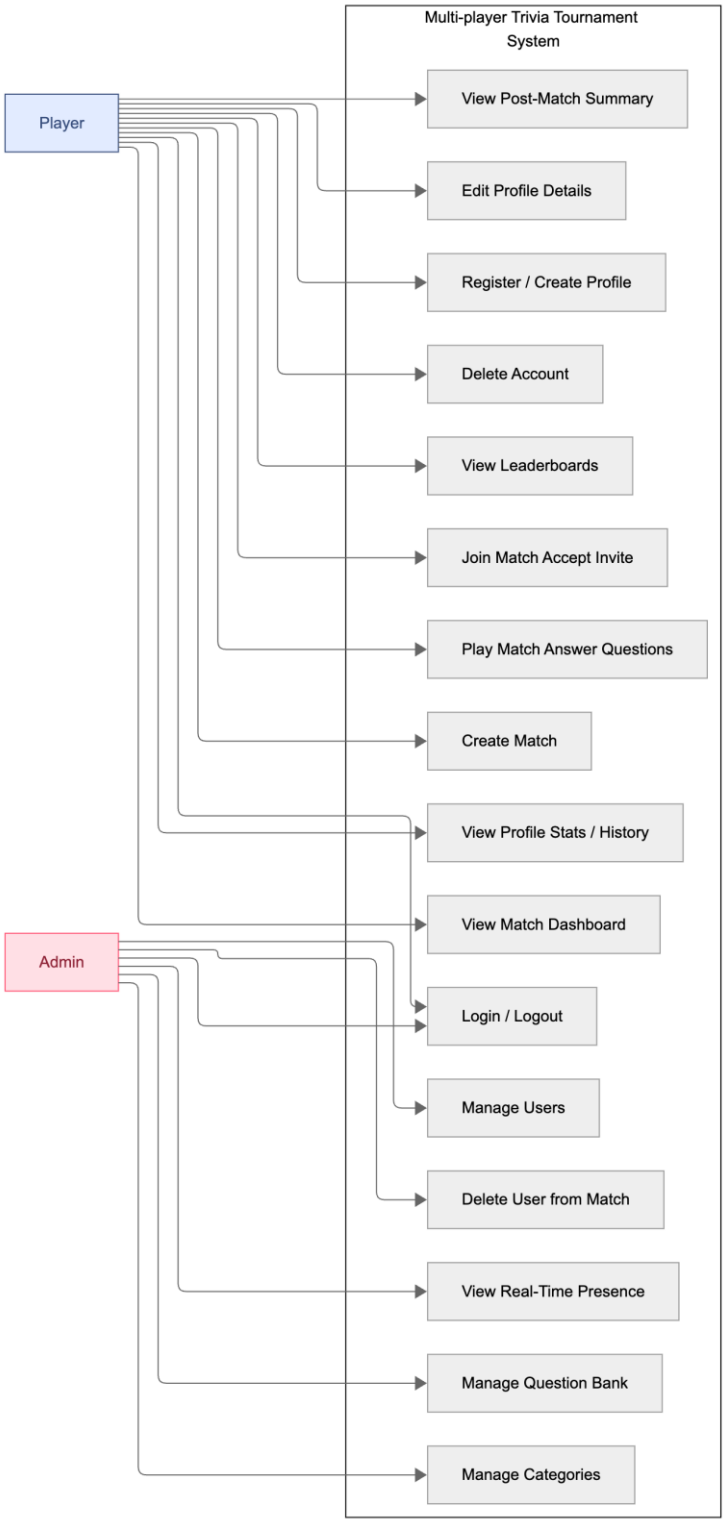
Frameworks and Tools Used:
Frontend: React (Vite 7+), Tailwind CSS
Backend: Node.js (v24+), Express, Socket.IO
Database: PostgreSQL with Prisma ORM
Authentication: JWT (access + refresh tokens), bcrypt for password hashing
Hosting / Deployment: Hetzner (Ubuntu) with Cloudflare Tunnel
Tutorials and Documentation Consulted (for learning only):
React – https://react.dev & Vite – https://vitejs.dev
Tailwind CSS – https://tailwindcss.com/docs
Socket.IO – https://socket.io/docs/v4
Node.js / Express – https://expressjs.com/
PostgreSQL – https://www.postgresql.org/docs
Prisma ORM – https://www.prisma.io/docs
Web scraping educational resources:
"What is Web Scraping and What is it Used For?" (YouTube, linked in project spec)
Scrapy Documentation – https://docs.scrapy.org/en/latest/

References

Academic and Technical Sources:
PostgreSQL 16 Official Manual
MDN Web Docs (JavaScript, Fetch API, WebSocket, DOM manipulation)
Open Trivia Database – used as a learning reference for trivia structure
Official documentation for React, Vite, Tailwind CSS, Prisma, and Express
Computer Science 343 – Project 2 Specification and lecture materials (2025

Appendices

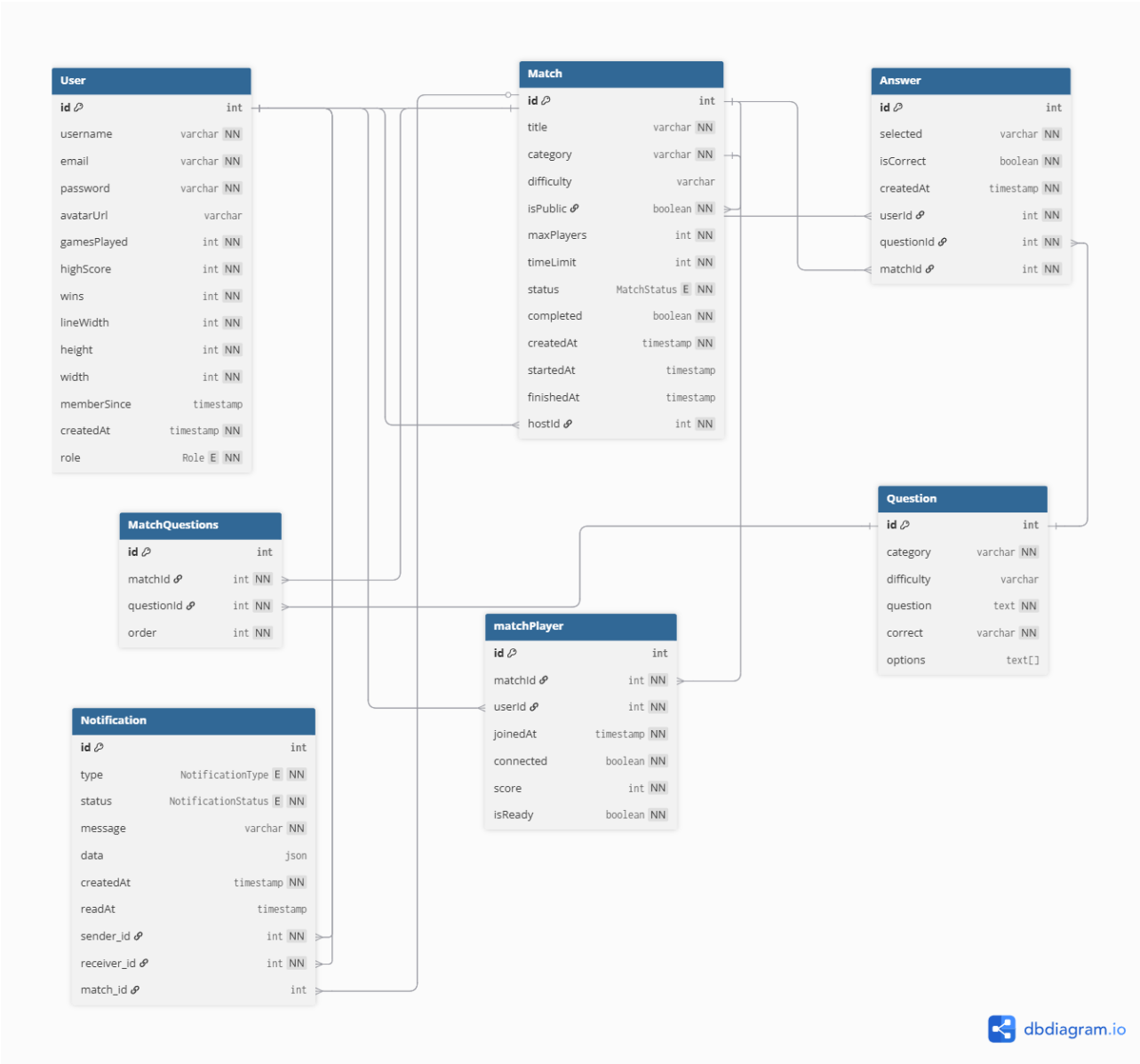
Appendix A: Use Case Diagram



(Figure A.1: System Use Case Diagram for Smartie Pants)

Appendix B: Entity Relationship Diagram

(Figure A.1: System ER Diagram for Smartie Pants)



Appendix C: Authentication Implementation

This appendix contains the core implementation details for *Smartie Pants* authentication, illustrating how JSON Web Tokens (JWT) and bcrypt are applied to secure user accounts and API access.

C.1 JWT Token Generation

Two tokens are generated for each authenticated user — an **access token** (30 minutes) and a **refresh token** (7 days). Each token is signed with a separate secret for added security.

```
const access = jwt.sign(payload, ACCESS_SECRET, { expiresIn: "30m" });
const refresh = jwt.sign(payload, REFRESH_SECRET, { expiresIn: "7d" });
```

- Access tokens authorize API requests.
- Refresh tokens issue new access tokens without re-login.
- Secrets are stored securely in environment variables:
  ACCESS_TOKEN_SECRET
  REFRESH_TOKEN_SECRET

C.2 JWT Middleware Verification

Protected routes verify the access token before processing any request:

```
const decoded = jwt.verify(token, process.env.ACCESS_TOKEN_SECRET);
req.user = decoded;
```

This ensures only authenticated users with valid roles (e.g., *admin*, *player*) can access restricted endpoints.

C.3 Password Hashing and Verification

Passwords are hashed before storage using bcrypt:

```
const hashed = await bcrypt.hash(password, 10);
```

Verification on login:

```
const valid = await bcrypt.compare(password, user.password);
```

- 10 salt rounds per hash
- Unique salt per password
- One-way hashing prevents reversal
- Slower hashing increases resistance to brute-force attacks

C.4 Database Security

In the database schema, only hashed passwords are stored:

```
Sql: password TEXT  -- bcrypt hash only
```

Unique constraints on username and email
Role-based access control ensures admin privileges are restricted

Together, these implementations ensure that *Smartie Pants* follows modern security best practices — short-lived tokens, encrypted secrets, and irreversible password storage — protecting both users and system integrity.

Appendix D: API Endpoints

This appendix lists all REST API endpoints for *Smartie Pants*, grouped by module. Each route follows the /api/... pattern and returns JSON data. JWT authentication is required for all routes unless stated otherwise.

Auth:

Method	Path	Description
POST	/api/auth/signup	Register a new user
POST	/api/auth/login	Login with username/email + password
POST	/api/auth/refresh	Refresh access token using refresh token

Users

Method	Path	Description
GET	/api/users/me	Get current user profile
PUT	/api/users/me	Update username, password, or avatar
GET	/api/users/search?q=	Search users by username

Uploads

Method	Path	Description
POST	/api/upload	Upload image and return file URL

Questions

Method	Path	Description
GET	/api/questions/all	Fetch all questions
GET	/api/questions/categories	Get distinct categories
GET	/api/questions/stats	Get question count by category
GET	/api/questions/random/:category/:count	Get random questions from a category
POST	/api/questions/import	Import from OpenTDB

POST	/api/questions/reset	Reset and reseed database
DELETE	/api/questions/:id	Delete single question
DELETE	/api/questions	Delete all questions

Matches

Method	**Path**	**Description**
POST	/api/matches	Create new match
GET	/api/matches	List all matches (filter/sort)
GET	/api/matches/:id	Get match details
POST	/api/matches/:id/join	Join match

Notifications

Method	**Path**	**Description**
GET	/api/notifications	Get user notifications
POST	/api/notifications/invites	Send game invite
PUT	/api/notifications/invites/:id/accept	Accept invite
PUT	/api/notifications/invites/:id/decline	Decline invite
PUT	/api/notifications/:id/read	Mark notification as read
DELETE	/api/notifications/:id	Delete notification

Appendix E: WebSocket Event Reference

This appendix lists all Socket.IO events used for real-time gameplay in *Smartie Pants*. It defines the communication flow, payloads, and event purposes for both client-to-server and server-to-client interactions.

Connection and authentication

Event Name	Direction	Payload / Parameters	Description
connection	Server → Client	–	Triggered when a client connects successfully to the Socket.IO server.
joinMatch	Client → Server	{ matchId, token }	Authenticates user via JWT and adds them to the specified match room. Creates an in-memory session if the match is not yet active.
disconnect	Client → Server	–	Fired when a user leaves or loses connection. The system automatically updates player lists and may pause or finish the match if the host disconnects.
leaveMatch	Client → Server	{ matchId }	Manually leaves a match and removes the player from the room.

Match Lifecycle

Event Name	Direction	Payload / Parameters	Description
startMatch	Client → Server	{ matchId }	Host-only event that starts the match. The backend validates host rights, loads assigned questions, and broadcasts matchStarted.
matchStarted	Server → Client	{ started: true }	Indicates to all players that the game has officially begun.
matchPaused	Server → Client	–	Sent when a host disconnects; temporarily pauses the game.
matchEnded	Server → Client	{ scores }	Broadcast when all questions have been completed or the match is terminated.

Question flow

Event Name	Direction	Payload / Parameters	Description
newQuestion	Server → Client	{ index, total, q, options, timeLimit }	Sends a new question with multiple-choice options and countdown timer.

requestCurrentQuestion	Client → Server	{ matchId }	Allows a reconnecting player to request the currently active question.
submitAnswer	Client → Server	{ matchId, answer, elapsedTimeMs }	Player submits their selected answer. The server validates correctness and updates scores.
answerSubmitted	Server → Client	{ correct, points, correctAnswer }	Returns immediate feedback to the player who answered.
questionResults	Server → Client	{ responses, scores, correctAnswer }	Broadcast after each question timer expires. Displays synchronized results and updated leaderboard to all players.
playersUpdate	Server → Client	{ matchId, players: [...] }	Periodically sent to update the list of active players and their scores.

Scoring and Timing

Feature	Description
Time-based scoring	Points are awarded using a dynamic formula: faster correct answers earn more points.
Synchronized timers	The server enforces uniform countdowns and only reveals results when the timer expires—preventing early reveals.
Automatic advancement	After each question, results are displayed for 5 seconds before the next question is broadcast automatically.
In-memory match store	Active matches are tracked in a Map keyed by match ID, containing player data, scores, timers, and question state.